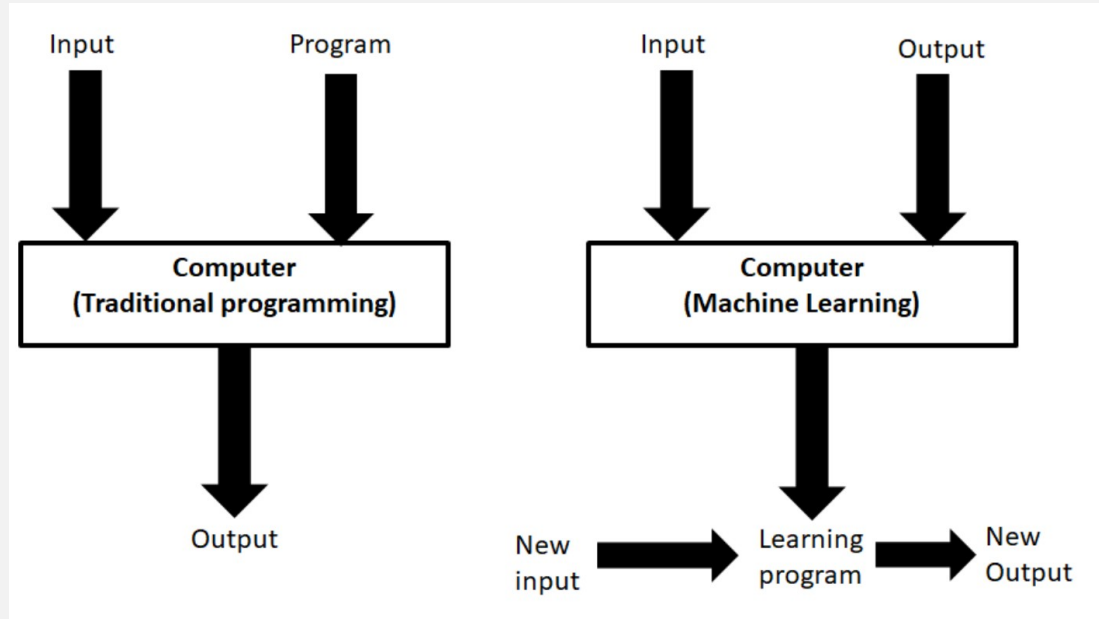


Software Engineering with Generative AI

Week 4 · Pre-Training, scaling law, fine-tuning, code tutorial
Instructor: Jie “JW” Wu
CS 5090 · Fall 2026
Michigan Technological University



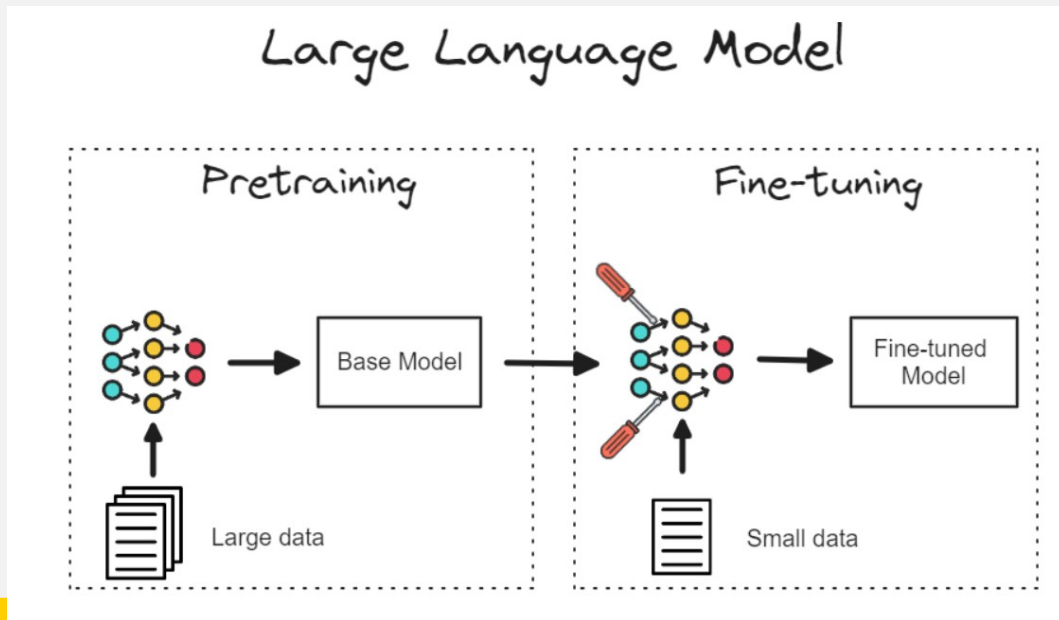
Traditional Learning Paradigm



Source: <https://nocomplexity.com/documents/fossm/whatism/whatism.html>

Modern Learning Paradigm

Pre-training + supervised training/fine-tuning

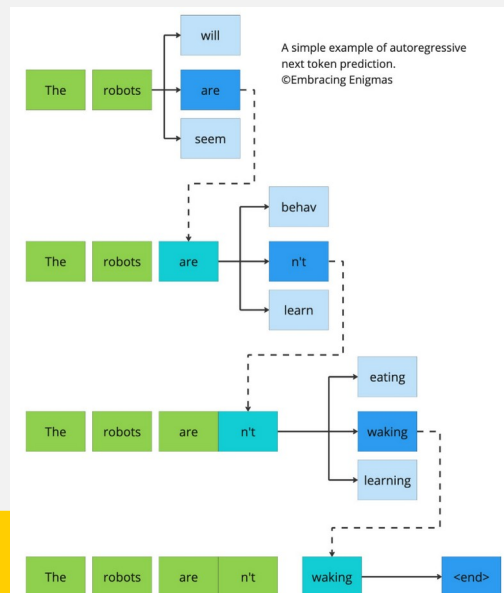
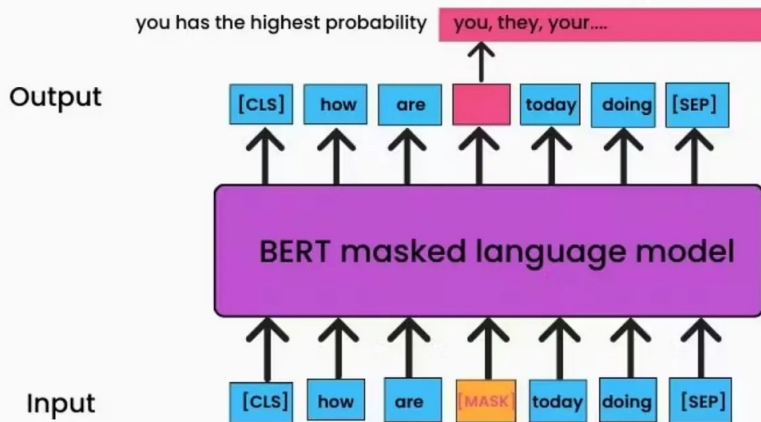


<https://www.linkedin.com/pulse/role-pretraining-fine-tuning-large-language-models-nikitha-r-9a-qdf/>

Pre-Training Objective

what labels or objectives used to train the vanilla Transformers?
During pre training, we have a large text corpus (no task labels)

- 1) Masked token prediction (BERT)
- 2) Next token prediction (GPT)



Pre-training architectures

Model Type	Pros	Cons	Use Cases
Encoder-Only	Strong contextual understanding, effective for text analysis tasks	Limited text generation capabilities, computationally intensive for large texts	Sentiment analysis, named entity recognition, text classification
Encoder-Decoder	Versatile, unified approach for various tasks, powerful text understanding and generation	Complex training, slower inference	Machine translation, text summarization, question answering
Decoder-Only	Excellent text generation, few-shot learning	Context limitations, resource intensive	Content creation, conversational AI, creative writing

<https://blog.gopenai.com/exploring-different-types-of-large-language-models-llms-encoder-only-encoder-decoder-and-7796ac663efa>

BERT

Idea. Keep only encoders + leverage encoded embeddings to make predictions.

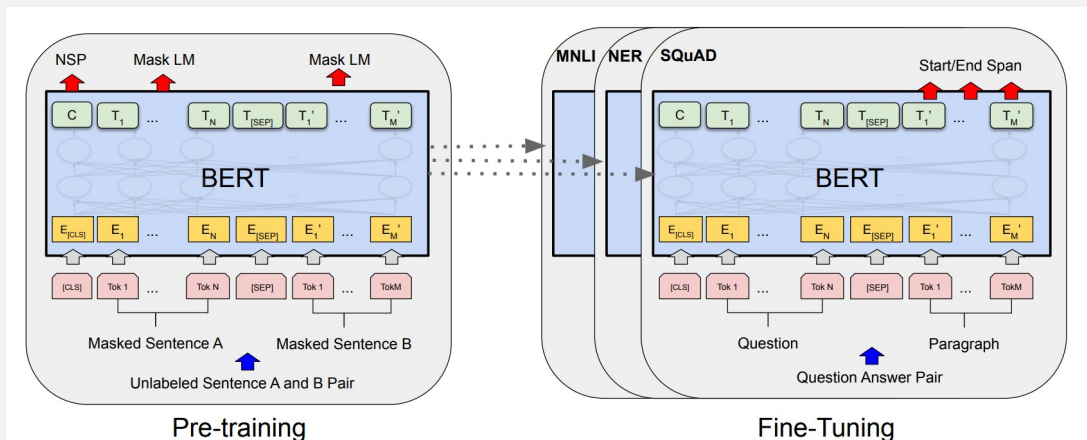


Figure 1: Overall pre-training and fine-tuning procedures for BERT. Apart from output layers, the same architectures are used in both pre-training and fine-tuning. The same pre-trained model parameters are used to initialize models for different down-stream tasks. During fine-tuning, all parameters are fine-tuned. [CLS] is a special symbol added in front of every input example, and [SEP] is a special separator token (e.g. separating questions/answers).

Pre-training Overview

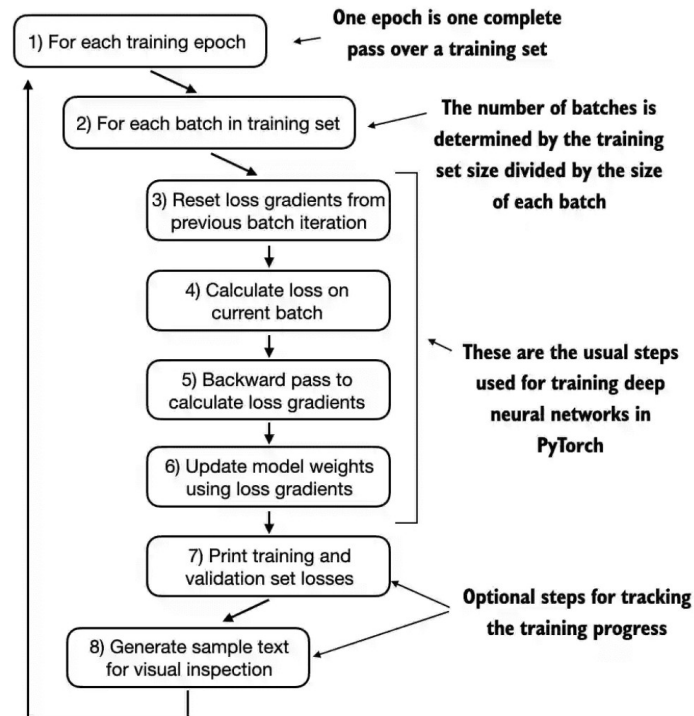
Goal: Learn pattern of text corpus

Objective function: Predict next token

Data mixtures: (data: trillions of tokens)

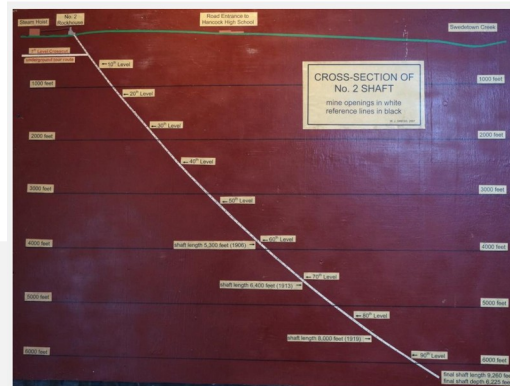
- Web-scraped (e.g. Wikipedia)
- Code (GitHub, StackOverflow)

Why coding tasks performs so well compared to other tasks?

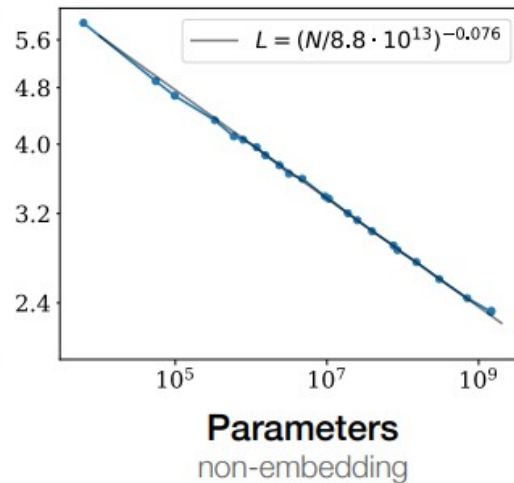
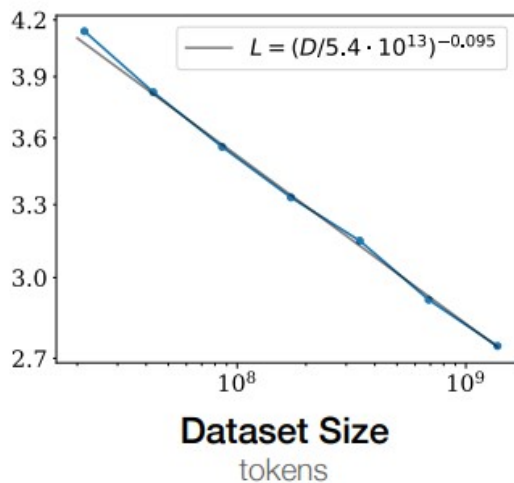
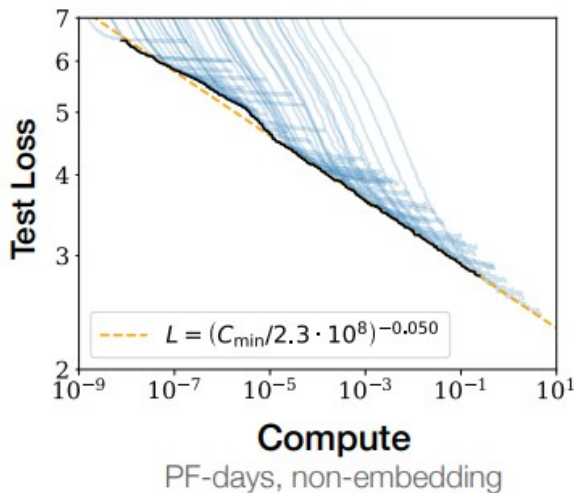


Scaling Law

why do industries like scaling law?



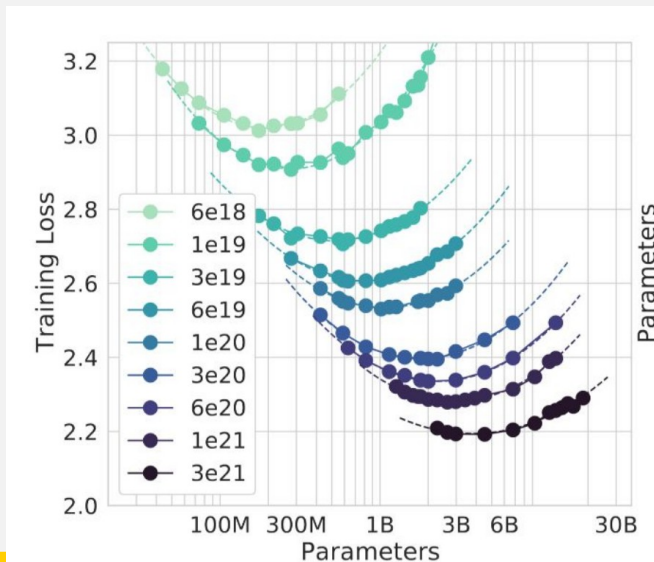
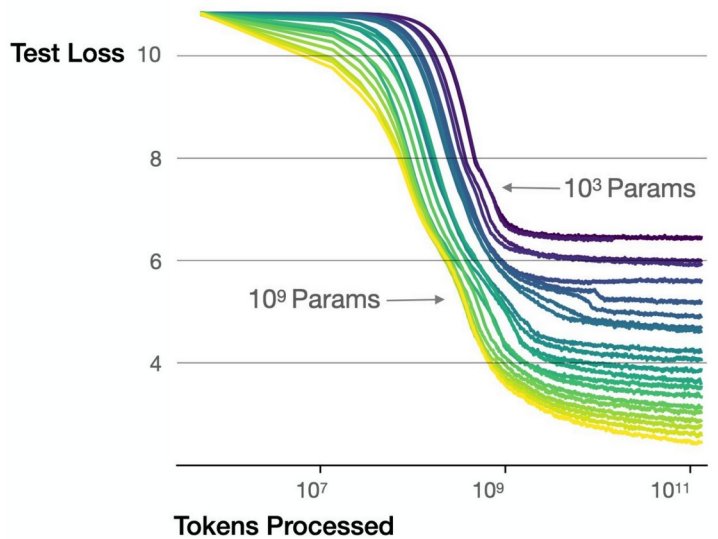
No 2 Shaft Diagram



Scaling Law

Sample efficiency

Chinchilla Law: D (#token) $\approx 20N$ (#para)



Pre-Training summary

- How does Pre-Training work.
- Pre-Training objectives and architectures.
- Understand Scaling law.

Why pre-training won?

What are the downsides of pre-training?

(hint: knowledge is learnt once in training, cost...)

Supervised Fine Tuning

Change the model behavior by tuning its weights

Strategy.

- Collect pairs of input/outputs with desired behavior (aka SFT data)
- Train using next word prediction objective given the input

Instruction Tuning

SFT on instruction-following data: "Instruction tuning"

Model is expected to be a assistant rather than a text completion machine

Data: human written data, synthetic data

Size: thousands to millions of examples

Pros and cons of instruction tuning?

Fine-Tuning Case Study

Problem: Code Models should increase the ability to ask clarifying questions for coding tasks, but how?

Key Observation:

Only {coding question, code} pairs exist in training data, causing LLMs to generate code regardless

- Such instruction tuning methods in Code LLMs **force** the models to generate code, regardless of any issues in the problem description.

Our Vision:

At the model level, LLMs **themselves** should have the ability to ask clarifying questions when needed

Example #1

```
#Fixed scalar addition
def increment_list(last):
    return [x + 5 for x in lst]

#Parameterized increment
def increment_list(lst, num):
    return [x + num for x in lst]

#Position-dependent increment
def increment_list(last):
    return [x + i for i, x in enumerate(lst)]

#Type-conditional increment
def increment_list(last):
    return [x+1 if isinstance(x, int) else x \
            for x in lst]
```

Ambiguous implementations for the problem statement **“Return list with elements incremented by a number”**. Variants demonstrate alternative interpretations of the problem.

Example #2



Q:
from typing import List

```
def candidate(numbers: List[float], threshold: float) -> bool:
    """ Check if in given list of numbers, are any two numbers meeting a
    certain condition (e.g. closer to or larger than each other than given
    threshold).
    >>> candidate([1.0, 2.0, 3.0], 0.5)
    False
    >>> candidate([1.0, 2.8, 3.0, 4.0, 5.0, 2.0], 0.3)
    True
    """
```

w/o ClarifyCoder

```
def has_close_elements(numbers: List[float], threshold: float) ->
bool:
    """ Check if in given list of numbers, are any two numbers closer
    to each other than
    given threshold.
    >>> has_close_elements([1.0, 2.0, 3.0], 0.5)
    False
    >>> has_close_elements([1.0, 2.8, 3.0, 4.0, 5.0, 2.0], 0.3)
    True
    """
    if len(numbers) < 2:
        return False
    seen = set()
    for num in numbers:
        close_to = [n for n in numbers if abs(num - n) < threshold and
n not in seen]
        if close_to:
            seen.update(close_to)
    return len(seen) > 0
```



w/ ClarifyCoder

A: The problem description is not clear. It mentions "closer to or larger than each other than given threshold", but it does not specify what exactly is meant by "closer".

Please clarify the definition of "closer" or "larger" in the context of the problem.



Asking Clarifying Questions All the Time?



User interaction with clarifying questions is enforced for standard coding tasks that could be completed without clarifications.

ClarifyGPT, TICODER, and ClariGen: explicitly require the user interaction module

Okanagan: asks clarifying questions only when necessary but, asks unnecessary questions, affecting pass@1



LLM-based workflows are complicated and costly than a single LLM.

ClariGen and Okanagan: need 3 calls to LLMs

ClarifyGPT: needs 4 calls



We need a balance between asking unnecessary questions and truly needed questions.

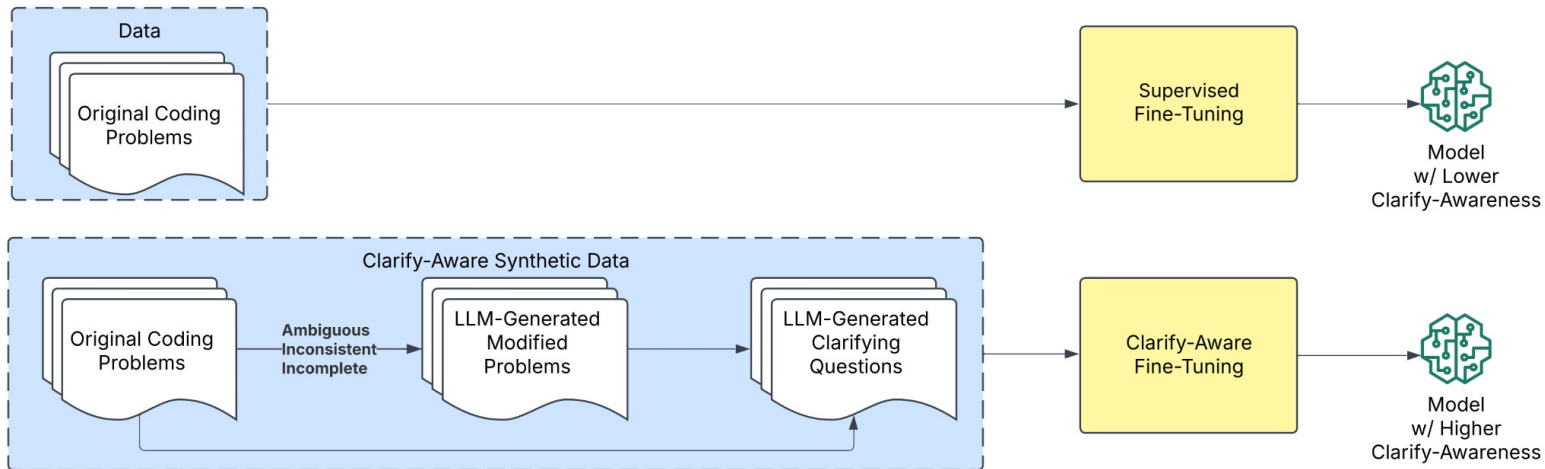
ClarifyCoder: Clarification-Aware Fine-Tuning



Goal: **alignment** for clarification-aware code generation.

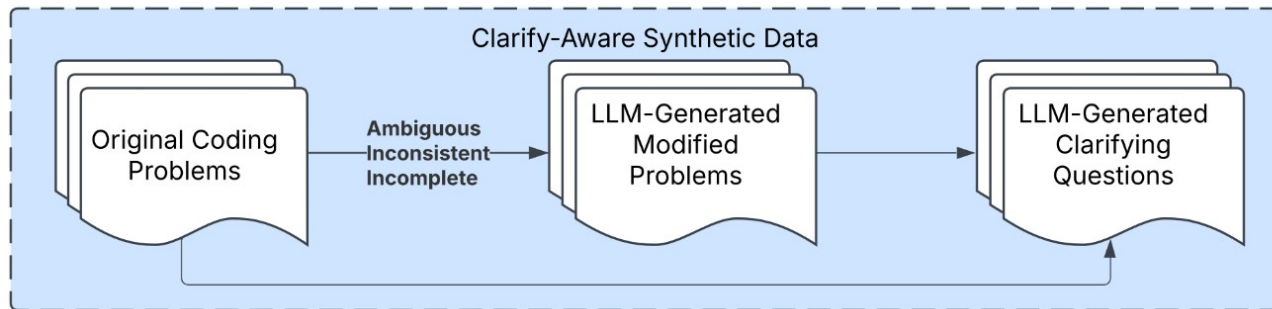
We propose **ClarifyCoder** that **fine-tunes** the Code LLMs with

- Synthetic Data Construction
- Clarification-Aware Instruction Tuning



Clarification-Aware Data Synthesis Process

Goal: produce ambiguous problem statements and their corresponding clarifying questions, thereby creating a robust dataset for training ClarifyCoder.



Clarification-Aware Data Synthesis Process

- Modify problems into the three categories
- Ask the model to generate a question to clarify the problem description
- Using APPS data, ~30 K new data are generated



Ambiguous

- Modification: Rewrite to introduce ambiguity by creating multiple valid interpretations or leaving key details unspecified.
- Question Gen: Identify points of ambiguity and formulate clarifying questions that resolve them.



Incomplete

- Modification: Rewrite to create incompleteness by omitting key concepts or conditions essential for solving.
- Question Gen: Identify points of incompleteness and formulate clarifying questions to resolve them.



Inconsistent

- Modification: Rewrite to introduce inconsistency by incorporating conflicting statements.
- Question Gen: Identify points of inconsistency and formulate clarifying questions to provide a consistent interpretation.

Clarification-Aware Fine-Tuning

We create a unified dataset D^{all} by merging $D^{clarify}$ and D^{og} :

$$D^{all} = D^{clarify} \cup D^{og}$$

We then perform instruction tuning on D^{all} by minimizing the combined loss function $\mathcal{L}^{all}$:

$$\mathcal{L}^{all} = - \sum_{t=1}^{|a^{all}|} \log P(a_t^{all} | a_{<t}^{all}, p^{all})$$

Example result:

Algorithm 1 Combined Instruction Tuning for Code Generation

Require: Pre-trained LLM

Require: D_{og} : Standard instruction tuning dataset

Require: $D_{clarify}$: Clarify-aware instruction tuning dataset

Require: r : Ratio $|D_{clarify}| / (|D_{og}| + |D_{clarify}|)$

Ensure: Instruction-tuned LLM

- 1: $D^{all} \leftarrow$ Combine D^{og} and $D^{clarify}$ according to ratio r and chosen sampling strategy
 - 2: **for** each sample $s \in D^{all}$ **do**
 - 3: Optimize the LLM on s by minimizing $\mathcal{L}^{all}$ (Equation 8)
 - 4: **end for**
 - 5: **return** Instruction-tuned LLM
-

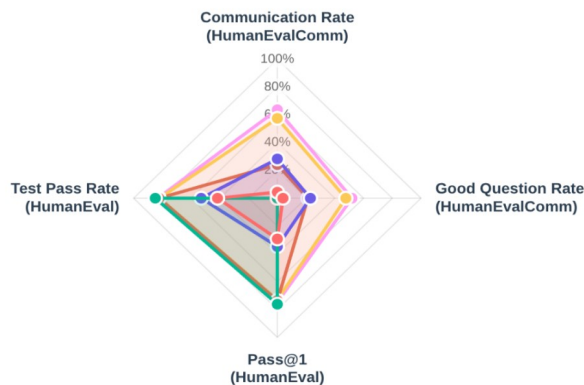
Ambiguous Problem Statement	DeepSeek Coder Response	ClarifyCoder Response
<pre>def candidate(l): """ candidate takes a list of integers as an input. it returns True if there are two distinct elements in the list that """</pre>	<pre>def candidate(l): seen = set() for num in l: if num in seen: return True seen.add(num) return False\n</pre>	It seems like your function description is incomplete. Could you please provide more details about what the function should return if there are two distinct elements in the list?

Performance Analysis (1 / 2)

Model	HumanEvalComm		HumanEval	
	Comm. Rate	Good Q. Rate	Pass@1	Test Pass Rate
CodeLlama	4.38%	3.93%	29.27%	41.63%
CodeQwen1.5 Chat	0%	0%	<u>76.22%</u>	<u>84.95%</u>
DeepSeek Chat	28.25%	23.03%	34.76%	53.00%
DeepSeek Coder	24.12%	21.29%	73.62%	82.36%
ClarifyCoder (w/ DS Coder)	<u>57.42%</u>	<u>47.68%</u>	73.01%	82.00%
ClarifyCoder (w/ DS Coder, loss=answer only)	<u>63.61%</u>	<u>51.93%</u>	<u>74.85%</u>	<u>83.29%</u>

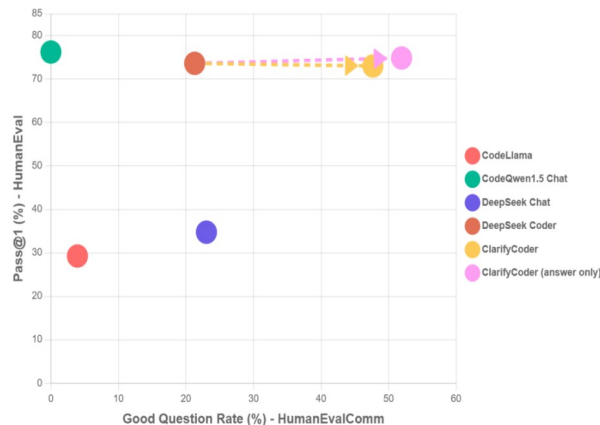
Table 3. Results on *HumanEvalComm* and *HumanEval* benchmarks for different LLMs. **First-place** and **Second-place** results are underlined and bolded, while **top 4** results are bolded. "loss=answer only" denotes the loss during fine-tuning, calculated only on the answers rather than on both the problems and the answers.

Performance Analysis (2 / 2)



● CodeLlama ● CodeQwen1.5 Chat ● DeepSeek Chat ● DeepSeek Coder
● ClarifyCoder ● ClarifyCoder (answer only)

(a) Multi-dimensional performance profile for clear (HumanEval) and ambiguous requirements (HumanEvalComm).



(b) a scatter plot of the trade-off between clarification-seeking ability and code generation performance. Dashed arrow represents the impact of Clarification-aware fine-tuning on the original DeepSeek Coder model.

Fig. 3. Comprehensive performance analysis of ClarifyCoder and baseline LLMs. "loss=answer only" denotes the loss during fine-tuning, calculated only on the answers rather than on both the problems and the answers.

Ratio r of standard and synthetic data.

- We changed the ratio r from 20% to 100%, with downsampling.
- Even with a modest 20% ratio, utilizing just 2,191 clarify-aware samples, we observed a notable improvement in clarify-awareness performance

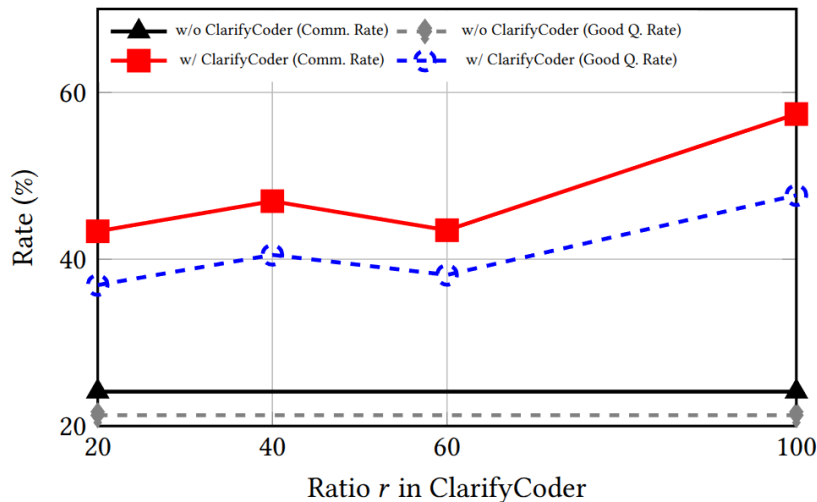


Fig. 6. ClarifyCoder performance with varying ratio r .

Comparison with Training-Free Methods

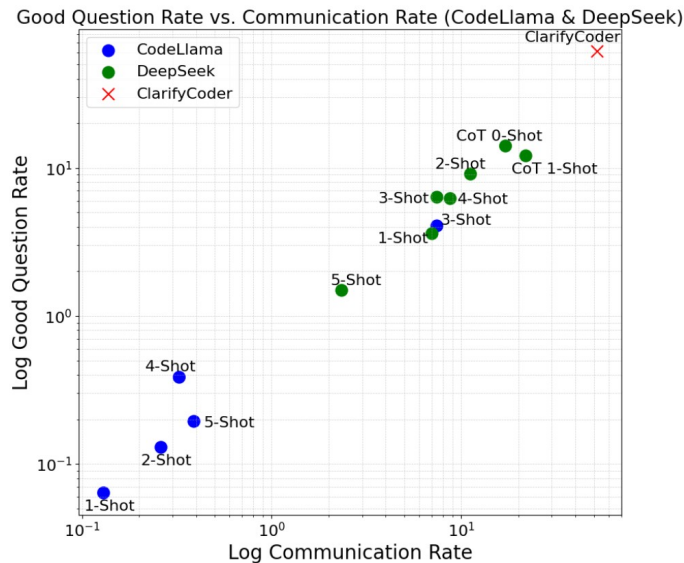


Fig. 4. Results on *HumanEvalComm V2* For In-Context Learning and Chain-of-Thought for CodeLlama and DeepSeek. The reported Communication rates and Good Question Rates are based on a log scale.

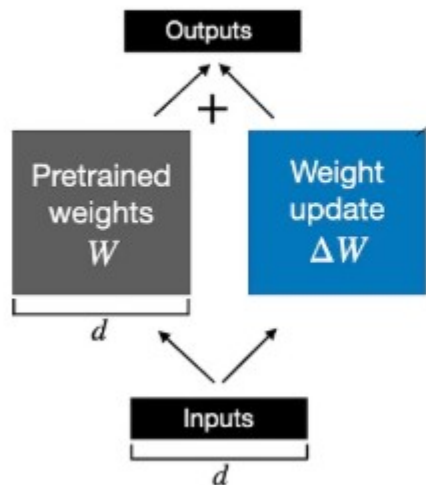
Compare with other LLM agents

Results of different methods on HumanEval benchmark (well-specified coding tasks). Pass@1 and Test PR measure code generated in the first (only) round. The top two results are bolded. Pass rates are considered unavailable for methods where user interaction to perform clarification is enforced.

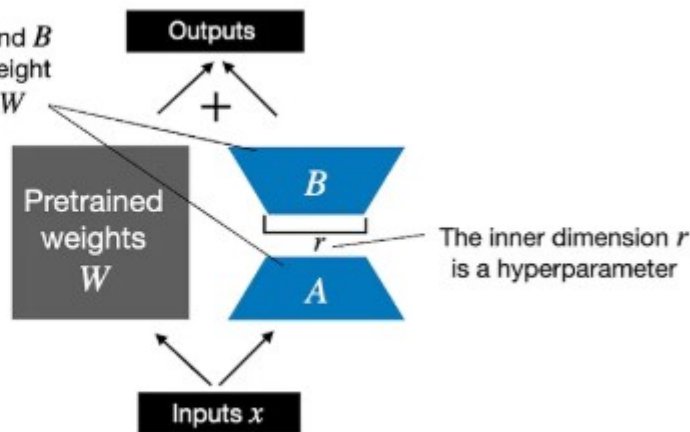
Method	Method Type	# LLM calls	Interaction Enforced	Pass@1	Test Pass Rate
TICODER	LLM Workflow	2	Yes	N/A	N/A
ClarifyGPT	LLM Workflow	4	Yes	N/A	N/A
ClariGen	LLM Workflow	3	Yes	N/A	N/A
Okanagan _(Base=ChatGPT)	LLM Workflow	3	No	27.45%	33.45%
Okanagan _(Base=DS Coder)	LLM Workflow	3	No	21.25%	24.3%
DeepSeek Chat	Single LLM	1	No	34.76%	53.00%
DeepSeek Coder	Single LLM	1	No	73.62%	82.36%
ClarifyCoder _(w/ DS Coder)	Single LLM	1	No	73.01%	82.00%
ClarifyCoder _(w/ DS Coder, loss=answer only)	Single LLM	1	No	74.85%	83.29%

Parameter-efficient finetuning (LoRA)

Weight update in **regular finetuning**



Weight update in **LoRA**



QLoRA

<https://watercrawl.dev/blog/LoRA-and-QLoRA>

Key Innovations

4-bit quantization

Instead of storing model weights in 16-bit or 32-bit floating point, QLoRA represents them with just 4 bits, dramatically shrinking memory use.

Double quantization

A second level of quantization reduces overhead from quantization constants, saving memory without major quality loss.

Paged optimizers

Inspired by virtual memory, QLoRA's paged optimizer stores optimizer states efficiently, swapping them in/out to reduce GPU memory spikes.

These innovations make QLoRA practical even for very large models, as noted in Databricks and Red Hat blogs.

Pretraining & fine-tuning in code

Pretraining:

https://github.com/rasbt/LLMs-from-scratch/blob/main/ch05/01_main-chapter-code/ch05.ipynb

NanoGPT in notebook

GPT (Zero to Hero), by AK:

https://colab.research.google.com/drive/1JMLa53HDuA-i7ZBmqV7ZnA3c_fvtXnx-

(Strongly recommend to watch the video and the code)